

BREAKPOINT V0 — Rapport technique officiel

Projet 7, V0 — casse-briques mobile rétro/moderne. Rapport rédigé à l'issue de la Phase 2 (condition éliminatoire) et de la Phase 3 (contenu V0), le 22 septembre 2026.

1. Identité du dépôt et état Git

Dépôt	https://github.com/scuillerjack-oss/breakpoint-v0
Branche de référence	main
HEAD (local = origin)	c5d2f1dd1680820f35ab22e8ce5b207400662541
git status	propre, aucune modification en attente
Isolation vis-à-vis d'autres projets	Dépôt Git indépendant, aucun code ni asset partagé avec HAVEN (scuillerjack-oss/haven-v0) ni RUP

2. Ce qui a été livré

Phase 2 — condition éliminatoire (moteur physique)

Collision « swept AABB » (algorithme David Glenn) : calcul du temps d'entrée exact dans $[0,1]$ pour chaque paire balle/collideur, résolu par sous-étapes (jusqu'à 8 par tick) pour gérer plusieurs impacts dans un même pas de temps. Simulation à pas de temps fixe (accumulateur, 60Hz) découplée du rendu, ce qui la rend déterministe et testable sans navigateur. Correction positionnelle réelle (clamp contre le rectangle du collideur touché) plutôt qu'un epsilon fixe, pour éliminer les balles collées même en cas de chevauchement profond. Angle de renvoi de la raquette influencé par le point d'impact ($\pm 60^\circ$ depuis la verticale selon la position relative sur la raquette).

Phase 3 — boucle de jeu V0

- 10 niveaux (courbe de difficulté progressive, briques standards + renforcées à plusieurs points de vie).
- 4 power-ups : **multiball**, **raquette XL**, **laser**, **balle perforante** — volontairement limité à 4 (jamais plus), en écho direct au retour joueur relevé sur un concurrent direct dans l'étude de marché (« trop de power-ups, interruptions trop fréquentes »).
- Pause intégrale avec décompte 3-2-1 à la reprise ; retour arrière navigateur/Android (popstate) ferme l'overlay de pause plutôt que de quitter la page.
- Tutoriels contextuels courts, déclenchés à la première occurrence de chaque mécanique.
- Sauvegarde locale (progression, tutoriels vus, réglages audio/haptique) persistée en localStorage, survit à un rechargement complet.
- Audio entièrement synthétisé (WebAudio, oscillateurs + enveloppes), aucun asset externe.
- PWA : manifest, service worker, icônes dédiées (motif de briques), installable.
- Mise à l'échelle « contain » d'une arène logique fixe (400×700) dans n'importe quel écran réel, canvas recalculé au redimensionnement/changement d'orientation, tenant compte du devicePixelRatio (plafonné à 2x).

3. Résultats de tests — complets, pas sélectifs

3.1 Suite unitaire (node:test)

✓ 27/27 tests verts : collision.test.js (6), simulation.test.js (15), levels.test.js (6).

2 bugs réels ont été trouvés PENDANT l'écriture des tests, pas masqués : (1) le signe de la normale de collision était incohérent selon l'axe touché — corrigé en la redéfinissant systématiquement comme opposée à la

composante de vitesse entrante sur l'axe gagnant ; (2) la correction positionnelle n'était pas assez robuste face à un chevauchement de départ profond — corrigée par un clamp réel contre le rectangle du collideur plutôt qu'un epsilon fixe. 3 fixturs de test ont aussi été corrigées car elles décrivaient des états de départ physiquement invalides (balle déjà enfoncée au-delà d'un collideur), pas des états réellement atteignables par la simulation.

3.2 Vérifications mobiles réelles (Playwright)

- ✓ Cycle tactile réel (glisser + lancer), aucune erreur console.
- ✓ Perte de focus (visibilitychange) → pause automatique.
- ✓ Redimensionnement/rotation ne casse pas le rendu (canvas correctement redimensionné).
- ✓ Sauvegarde persistée après rechargement complet (progression + réglages).
- ✓ Aucun débordement horizontal à 5 largeurs testées (320/360/390/414/480px).

3.3 Performance sous stress

Scénario : niveau normal (grille de briques complète) + 6 balles ajoutées simultanément (7 balles au total) via un scénario de test dédié. Résultat : FPS moyen $\approx 59,8$, FPS minimum $\approx 8,6$ (un seul creux transitoire isolé), 1 frame sur 235 sous 50fps, 0 erreur console. Le rendu ne s'effondre pas techniquement sous ce scénario extrême.

3.4 CI réelle (GitHub Actions, pas seulement l'environnement de développement)

✓ Premier run réel (id 35690584929, commit 8fc5a9e) : le job build a réussi intégralement dans le runner Ubuntu de GitHub — npm ci, npm test (27/27), build de production, installation Chromium, et les 5 checks mobiles Playwright, tous verts. Confirmation indépendante de l'environnement local.

■ Le job deploy a échoué : « Failed to create deployment (status: 404)... Ensure GitHub Pages has been enabled ». Cause identifiée : pour un dépôt neuf, GitHub Pages doit être activé une fois manuellement (Settings → Pages → Source → « GitHub Actions »). Ceci est une action du propriétaire du dépôt — aucun outil disponible dans cette mission ne peut la faire à sa place. Le code est sain et sécurisé sur origin/main ; seule la mise en ligne effective est en attente de cette action.

4. Ce qui N'A PAS été validé — limite honnête, explicitement demandée

■ Le cahier des charges est explicite : une suite de tests verte NE SUFFIT PAS à valider la sensation de jeu réelle. Ce rapport le confirme sans détour : la fluidité perçue à la main, sur un vrai téléphone, sous une vraie latence tactile, sur un vrai écran, reste ENTièrement À CONFIRMER par une bêta réelle. Tout ce qui précède (tests unitaires, Playwright, CI) élimine des classes entières de bugs (tunneling, balle collée, incohérence physique, régressions), mais ne remplace pas ce jugement humain.

- Bouton retour matériel Android : le mécanisme web (popstate) est en place mais n'a jamais été testé sur un vrai bouton retour matériel — seulement son équivalent navigateur.
- Latence tactile réelle sur téléphone physique (vs Chromium headless) : non mesurée.
- Ressenti du power-up laser/perforant en contexte réel de jeu (équilibre) : jugé fonctionnellement correct par les tests, jamais jugé « amusant » par un vrai joueur.
- Déploiement GitHub Pages : le code est prêt, la mise en ligne effective attend l'action manuelle décrite en section 3.4.

5. Décisions en attente de validation du porteur du projet

■ Étude de marché (docs/BREAKPOINT_Etude_Marche_V0.pdf) : à lire avant toute décision de positionnement (ASO, mots-clés) — attention notable à la confusion de genre avec Block Blast (puzzle de blocs, pas un casse-briques).

- Étude économique (docs/BREAKPOINT_Etude_Economique_V0.pdf) : à lire avant toute implémentation de monétisation. Conclusion du modèle : aucun scénario réaliste sans acquisition payante ne justifie d'investir sur la monétisation avant validation du cœur de jeu par une bêta réelle.
- Activation de GitHub Pages (Settings → Pages → Source → GitHub Actions) — seule action bloquant la mise en ligne, décrite en détail dans README_REPRISE_BREAKPOINT.md section 4.
- Trajectoire Capacitor/Android/AAB (README_REPRISE_BREAKPOINT.md section 7) : documentée, non engagée — à valider avant tout investissement de temps sur l'emballage natif.

6. Conclusion

La question posée en priorité absolue par le cahier des charges — « le cœur du casse-briques est-il suffisamment bon pour justifier de continuer le projet ? » — reçoit une réponse techniquement positive : aucun défaut de tunneling, de balle collée, ou d'incohérence physique n'a résisté aux tests, y compris sous stress (multiball + grille complète) et dans l'environnement CI réel de GitHub, pas seulement en local. La Phase 3 a été construite par-dessus ce socle sans compromettre la fluidité (aucun effet n'a été ajouté au détriment du framerate). Mais conformément à la règle de décision du cahier des charges, cette conclusion reste techniquement fondée et humainement non confirmée : la validation finale appartient à une bêta réelle sur téléphone physique, pas à ce rapport.